



CI/CD

Scaling & Performance
Optimization



Outline

1 Performance Tuning

1

Performance Tuning



Apa itu Performance Tuning pada K8S

- Proses untuk mengoptimalkan kinerja workload dan cluster Kubernetes.
- Fokus pada resource utilization, response time, throughput, dan cost-efficiency.

Area umum yang biasa di Tuning

- Pod Resource Management
- Node Sizing & Scheduling
- Auto-scaling Mechanism
- Network Performance
- Storage IOPS & Latency
- Image Size & Start-up time

1

Tips Tuning pada K8S



Gunakan Resource Limit



Resource Requests dan Limits adalah cara Kubernetes mengatur alokasi **CPU** dan **memori (RAM)** untuk setiap **container** di dalam Pod, agar pemakaian sumber daya lebih **terprediksi, efisien, dan stabil**.

- **Requests:** Minimum resource yang dibutuhkan
- **Limits:** Batas maksimum yang diperbolehkan

Gunakan HPA (Horizontal Pod Autoscaler) untuk workload dinamis

Gunakan Resource Limit

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "128Mi"
        cpu: "250m"
      limits:
        memory: "256Mi"
        cpu: "500m"
```


Mengapa Penting ?

- Mencegah satu container memakan seluruh resource node.
- Membantu scheduler memilih node dengan tepat.
- Menghindari crash atau ketidakstabilan sistem.
- Wajib dalam cluster production untuk efisiensi dan keamanan.

Tanpa **requests** dan **limits**, Pod bisa:

- Menyebabkan **resource starvation** (kelaparan resource)
- Tidak diprioritaskan oleh scheduler
- Rentan menyebabkan **node instability**

Gunakan Tiny Base Image



Sangat penting dalam memilih base image yang akan digunakan pada K8S deployment, ada beberapa tipe umum base image seperti berikut

- **alpine** (~5MB)
- **distroless** (tidak punya shell atau package manager)
- **busybox** (ultra-minimal)

Mengapa Penting ?



Pemilihan **base image yang tepat sangat penting** karena base image adalah **fondasi** dari seluruh container image kamu. Kesalahan memilih bisa menyebabkan:

- Ukuran image membengkak
- Keamanan lemah (banyak CVE)
- Performance buruk
- Proses CI/CD lambat
- Sulit dimaintain di production

Berikut adalah **alasan pentingnya memilih base image dengan tepat**:

Mengapa Penting ?

1. Keamanan (Security)

- Base image yang salah bisa membawa **kerentanan (CVE)** ke dalam aplikasi.
- Image usang = lebih banyak bug & exploit.
- Gunakan base image **yang aktif dipatch**, misalnya:
 - `alpine`, `debian:bullseye`, `distroless`

2. Ukuran Image dan Kecepatan Deploy

- Semakin kecil image → semakin cepat proses:
 - Build
 - Pull dari registry
 - Deploy ke Kubernetes

Mengapa Penting ?

3. Performa dan Efisiensi Resource

- Base image ringan dan minimal tidak membawa proses/proxy/daemon tambahan yang membebani CPU/memori.
- Cocok untuk autoscaling di Kubernetes → lebih murah & responsif.

4. Debugging dan Maintainability

- Image yang terlalu minimal (misalnya **distroless**) **tidak punya shell** → susah debugging jika tidak disiapkan dari awal.
- Pastikan base image sesuai kebutuhan: development vs production.

Aktifkan Health Check



Health check di Kubernetes adalah **mekanisme otomatis** untuk memastikan bahwa aplikasi (Pod) yang berjalan **sehat dan bisa menerima trafik**, serta untuk memutuskan apakah Pod perlu **direstart** atau **ditunda dari load balancing**.

Health check dilakukan melalui dua jenis probe utama:

→ livenessProbe

→ readinessProbe

Aktifkan Health Check

1. Liveness Probe

Mengecek **apakah container masih hidup** (bukan crash atau hang).

Jika gagal, Kubernetes akan **restart** container secara otomatis.

Contoh kasus: aplikasi stuck, deadlock, infinite loop.



```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

Aktifkan Health Check

2. Readiness Probe

Mengecek **apakah container siap menerima trafik**.

Jika gagal, Pod akan **dikeluarkan sementara dari Service (load balancer)**.

Tidak menyebabkan restart.



```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
```

Aktifkan Health Check



Jenis Metode Health Check

- **httpGet** → akses URL endpoint (`/healthz`, `/ready`, dll.)
- **exec** → jalankan perintah di dalam container (`cat /tmp/healthy`)
- **tcpSocket** → buka koneksi TCP ke port tertentu

HPA atau **Horizontal Pod Autoscaler** adalah fitur Kubernetes yang secara otomatis **menambah atau mengurangi jumlah Pod** dalam sebuah Deployment, StatefulSet, atau ReplicaSet **berdasarkan beban kerja (workload)** seperti:

- CPU usage
- Memory usage (dengan metrics-server)
- Custom metrics (dengan Prometheus, KEDA, dll.)

Gunakan HPA



Tujuan HPA

- Menjaga **kinerja aplikasi tetap optimal**
- **Menghemat resource** saat trafik turun
- Meningkatkan **resiliensi dan skalabilitas otomatis**

Gunakan HPA

- **Mengamati** beban pod (misalnya CPU > 70%)
- **Menghitung** jumlah pod ideal berdasarkan rumus internal
- **Men-scale** jumlah Pod sesuai kebutuhan (min-max)

Syarat Agar HPA Berfungsi:

- **metrics-server** harus aktif di cluster
- Container memiliki **resource requests** (CPU/mem)
- Versi Kubernetes minimal 1.6 (untuk HPA v2)

Gunakan HPA



```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 75
```

Field

Arti

averageUtilization

Target penggunaan rata-rata CPU/memory (dalam persen dari requests)

minReplicas / maxReplicas

Rentang autoscaling yang diperbolehkan

Dua metric aktif

HPA akan menskalakan berdasarkan **metric yang paling membutuhkan** scaling saat itu (bukan gabungan rata-rata)

A background image showing three business professionals in an office. A man in a suit and glasses is standing and smiling, with his arms raised in a celebratory gesture. Two women, also in business attire, are seated at a desk with a laptop, looking up at him and smiling. The entire scene is overlaid with a dark teal color filter.

</Rakamin

Terima kasih